

Entropy Code or Memory Layout? A Negative Result in Lossless BF16 Weight Compression, and the 8-bit Block Index It Led To

Arnau Ferrerons Manich

Independent researcher · ORCID 0009-0002-7245-7221

Code, data and logs: <https://github.com/ArnauFerma/bf16-exponent-compression>

Archived: doi:10.5281/zenodo.22736348

September 2026

Abstract

Entropy-coding the exponent field of BF16 weights, as in DFloat11, losslessly removes about a third of the bytes at the cost of a decompression kernel on every forward pass. This report tests whether a fixed-shape prefix code with a 10-byte table (a “ladder” code) decodes faster on a GPU than canonical Huffman with a 4 KiB lookup table, for under one point of compression. It does not: on Pascal, Ampere and Ada the ladder code loses 0.86 points and is never faster where it would be run. The test produced something that does work. Replacing the per-block uint32 offset index with 8-bit block lengths recovered by a warp prefix-sum cuts the index from 4 to 1.125 bytes per block (0.14 bits/weight, against about 0.21 for DFloat11’s 5-bit per-thread gaps), adds 2.25 points of compression, and costs nothing measurable on any card. With the output coalesced through shared memory, the reference decoder becomes 2.0x faster and 2.25 points smaller, and decodes 64M symbols in 0.20 ms on an RTX 4090. Exact measurements on Qwen3-0.6B place canonical Huffman 0.033 bits/weight from the exponent-only floor and 0.100 from the true floor of the 16-bit symbol; the field split gives away 0.076 bits/weight. A 16x collapse at large block sizes on Pascal is an L2 effect that shrinks to 2x and 1.7x on cards with more L2 per thread, and the optimised decoder’s time tracks SM clock, not bandwidth. Every number traces to a committed log published with the code.

1 Introduction

Large language models are served in bfloat16, and the 16 bits are not equally informative. Sign and mantissa of trained weights are close to incompressible; the 8-bit exponent of a roughly Gaussian weight carries under 3 bits of entropy.

DFloat11 [1] exploits this: Huffman-code the exponents, store sign and mantissa raw, decompress on the GPU before each matrix multiply. Outputs are bit-exact and the model is ~30% smaller. The price is decode time — 1.4–2x slower inference than BF16 at batch size 1.

Because the decoder runs on every forward pass, the entropy code looked like the place to find speed. Canonical Huffman needs a table and costs at least one shared-memory access per symbol. A prefix code with a *fixed shape* — count leading ones, read a small index — needs a 10-byte table and decodes with a `clz`, a branch and a shift.

The hypothesis, recorded before any measurement: such a code decodes faster than canonical Huffman on a GPU, giving up under one point of compression.

It does not. This report describes the test, and what the test found instead. The contributions, in the order they matter to a reader who wants to use them:

1. **An 8-bit block index with a warp prefix-sum** (§5.4). 1.125 instead of 4 bytes per block, +2.25 points of compression, zero measurable cost on Pascal, Ampere and Ada. It dissolves the trade-off between block size and index overhead in a fixed-symbols-per-block design, and it costs 0.07–0.14 bits/weight against about 0.21 for DFloat11’s per-thread gaps, with a single decode pass instead of two.
2. **The negative result** (§5.2, §5.6). The ladder code is 0.86 points worse in rate and, once the decoder’s memory access pattern is fixed, between tied and 1.36x slower than Huffman at every configuration one would run.
3. **Where the decoder’s time goes** (§5.3, §5.6). Coalescing the output write through shared memory is worth 1.9x on Pascal and 4.6x on Ada. A 16x collapse at large block sizes is an L2 effect that decays with L2 per resident thread. After both fixes the decoder’s time follows SM clock, not bandwidth.
4. **The rate floor, measured exactly** (§5.5). Neighbouring exponents are independent; exponent and mantissa of the same weight share 0.040 bits; the field split gives away 0.076 bits/weight against a full-alphabet code.

All predictions were written down before the measurements that tested them and are kept in the repository as written. The ones that failed are reported as failures.

2 Background

BF16 fields. [1 sign][8 exponent][7 mantissa]. On the 596,049,920 unique weights of Qwen3-0.6B: sign 1.000 bits of entropy, mantissa 6.973 of 7, exponent 2.645 of 8. Only 43 exponent values occur; three of them carry 72% of the mass. Everything compressible is in the exponent.

Canonical Huffman and the block index. A Huffman code [4] in canonical form [5] is specified by its code lengths alone, so the table costs at most 256 bytes. GPU decoding uses a primary lookup table indexed by the next k bits ($k = 11$ here: $2048 \times 2 \text{ B} = 4 \text{ KiB}$ in shared memory) with a canonical search for the rare longer codes.

Variable-length codes destroy random access. The remedy used here is a coarse index: one absolute bit offset per block of BLOCK symbols. Blocks decode in parallel, one thread each; inside a block the decode is serial. BLOCK trades index overhead against the length of that serial chain.

DFloat11 solves the same problem from the other side (from its published kernel, `decode.cu`): each thread owns a fixed 8 bytes of compressed stream and a 5-bit “gap” says how many bits to skip to the first whole codeword — about 0.21 bits/weight of index on this data. Symbols per thread then vary, so the kernel decodes twice: a counting pass and a block-wide prefix-sum in shared memory give each thread its output position, then the real pass writes BF16 values, sign and mantissa already merged, through shared memory in coalesced form. Fixed symbols with an input-side prefix-sum, or fixed bits with an output-side one: the two designs are the two corners of the same trade-off.

The ladder code. Read ones until the first zero; the count selects a rung; read that rung’s index bits into a small table. With rungs (1, 1, 1, 2): lengths 2, 3, 4, 6 for the 10 most frequent symbols, and a 12-bit escape (1111 + raw exponent) for the rest. Kraft’s sum is exactly 1; the code is complete. The shape is fixed, so only 10 symbol bytes change between models. Decoding is `__clz(~w)`, a four-way branch, a shift and a mask.

Related work. DFloat11 [1] (NeurIPS 2025) is the direct reference. In June 2026, Tan et al. [2] (ISCA 2026) reported up to 11x its throughput by coding the whole 16-bit symbol with rANS and fusing decompression into the GEMM, so decompressed tiles never touch global

memory. Its only stated objection to Huffman is the rate gap from integer-length codes; it does not claim — and it would be wrong to claim — that Huffman cannot do tile-granular random access. Its tiles get random access the way blocks do here: independent streams plus an offset table. §6 returns to [2].

3 Hypothesis, and predictions recorded before measurement

From the repository’s first commit, before any GPU was available:

A ladder prefix code with a 10-byte table decodes faster on a GPU than canonical Huffman with a 4 KiB hierarchical LUT, at a cost of under 1 point of compression.

The same document recorded why it might fail — “Canonical Huffman uses a LUT: one SRAM access. The ladder uses `clz` + branch + shift + mask. On instruction count it may lose” — and named the most likely outcome: “the ladder does not gain any speed and ends up a curiosity 0.8% worse than Huffman”.

It also stated the question that would decide the matter: is the decode kernel memory-bound or compute-bound? In a memory-bound kernel the entropy code cannot matter.

Later predictions, each written before the measurement that tested it, are quoted in §5 where they apply, including the ones that failed.

4 Experimental setup

Everything not repeated here is in `METHODOLOGY.md` in the repository.

4.1 Data

Qwen/Qwen3-0.6B [6], `model.safetensors` (SHA-256 `f47f7117...6874b`), all tensors BF16. The file stores `lm_head.weight` and `model.embed_tokens.weight` as separate, bit-identical tensors; the extractor drops exact duplicates, leaving **596,049,920 unique weights**. Weights are read as raw `uint16` bit patterns and never converted to float.

Two samples are used besides the whole model:

- a random 2M weights, with replacement, for the CPU bit-exact roundtrip of each codec (tables always come from the whole-model histogram; a prefix code is memoryless per symbol, so the sample exercises every code path including the escape);
- the **first 64M weights** for every GPU measurement.

The 64M lie inside the embedding matrix, which compresses slightly better than the model average (exponent entropy 2.552 vs 2.645 bits). Compression figures from GPU runs are therefore sample figures and the whole-model projection is given alongside. Timing conclusions do not depend on the distribution.

4.2 Hardware

	GTX 1050 Ti	A100-SXM4-80GB	RTX 4090
architecture	Pascal, SM 6.1	Ampere, SM 8.0	Ada, SM 8.9
SMs / L2	6 / 1 MiB	108 / 40 MiB	128 / 72 MiB
L2 per resident thread	85 B	190 B	384 B
peak bandwidth	112 GB/s	2039 GB/s	1008 GB/s
SM clock during runs	139–1923 MHz, unlocked	1140 MHz	2520 MHz
where	author’s PC (Windows)	rented container	rented container

The design was developed on the 1050 Ti. The two rented cards ran identical code and sample, unattended, after the predictions of §5.6 were recorded. Nsight Compute does not support Pascal and its counters were blocked in both containers: occupancy, stalls and divergence are unmeasured on every card.

Software: Python 3.12, NumPy 2.5.3, CuPy 14.2.0, CUDA 12.9; kernels are CUDA C compiled at run time through NVRTC.

4.3 Protocol

Correctness before timing. The vectorised encoder is verified byte-for-byte against the reference bit writer. Every kernel is verified bit-exact against the source exponents — 32M symbols, all 20 BLOCK $\times$ threads combinations — before any timing; the setup scripts refuse to proceed otherwise, and each benchmark harness re-checks every configuration it times. Every row reported here verified.

Timing. CUDA events around one launch; 0.6 s of warm-up per configuration; 11 repetitions, median reported, IQR published beside it; configuration order shuffled with a fixed seed; SM clock sampled around each measurement. IQR was 0–3 ms on the unlocked 1050 Ti and 0 on every row of the rented cards. A full re-run on the 1050 Ti reproduced every ratio and conclusion, absolute times moving 1–6%.

The memory-floor kernel reads exactly the words each decoder thread would read and writes the same bytes, but does no decoding. It is a ceiling for *that access pattern*, not for the hardware.

Attribution. All combinations of candidate optimisations are compiled and measured separately. When an optimisation moves the bottleneck, earlier comparisons are re-run, not extrapolated.

Compression accounting. Exact, from symbol counts, in bits per weight:

$$8 + \frac{\text{exponent bits}}{\text{BLOCK}} + \frac{8 \cdot \text{index bytes per block}}{\text{BLOCK}},$$

plus a 10 B (ladder) or 256 B (Huffman) table.

5 Results

5.1 Rate

whole model, BLOCK=256, uint32 index	bits/exponent	reduction
exponent entropy	2.645	—
canonical Huffman	2.678	32.48%
ladder (1,1,1,2)	2.815	31.62% (−0.86 pts)
ladder (1,1,2,2)	2.847	31.42%
ladder (1,2,2,3)	3.063	30.07%

The shape chosen on synthetic Gaussian weights stays the best of the three on the real model, and the gap is within the pre-registered budget of one point. The rate half of the hypothesis holds. Both codecs roundtrip bit-exactly.

5.2 GPU decode: is it memory-bound?

Two kernels with an identical interface — same index, same one-thread-per-block mapping, same window reader, same output layout — differing only in the symbol decoder, plus the memory-floor kernel. GTX 1050 Ti, 64M symbols, base kernel, 128 threads:

BLOCK	Huffman	ladder	floor	ladder/Huffman
64	9.70 ms	9.75	9.57	1.006
128	52.11	34.96	12.77	0.671
256	84.86	58.22	13.35	0.686
512	121.08	91.19	16.42	0.753
1024	157.16	122.29	20.36	0.778

At BLOCK=64 — the only operating point one would choose — both decoders sit within 2% of the floor and within 1% of each other. The entropy code is irrelevant; the ladder is marginally slower because its stream is 4.8% larger.

At BLOCK $\geq$ 128 the ladder wins by up to 33%, the compute-bound regime the hypothesis anticipated. Those configurations are 3–16x slower in absolute terms.

Three caveats bound this:

- “Memory-bound” here means bound by a bad access pattern. The best configuration reaches 9 GB/s, 8% of peak.
- Output dominates. Of 88.7 MB moved at BLOCK=64, 64 MB (71%) is the one-byte-per-symbol output; the entropy code can touch at most 23% of the traffic.
- The collapse between BLOCK=128 and 256 coincides with the resident working set (520 $\rightarrow$ 1040 KiB) crossing the 1 MiB L2. Recorded as consistent, not proven; it became the prediction of §5.6.

5.3 Attribution: input versus output

Each thread walks its own bit region, so a warp reads 32 scattered streams. Each thread writes BLOCK consecutive bytes, so a warp’s 32 stores hit 32 distinct sectors. Both have the same fix:

the threads of one CUDA block own a contiguous range, so stage it through shared memory and move it coalesced. Four variants were compiled so the gain could be attributed rather than guessed. BLOCK=64, 128 threads:

variant	1050 Ti	A100	4090
base	1.00	1.00	1.00
input staged	1.01x	1.00x	1.02x
output staged	1.92x	2.61x	4.55x
both staged	1.48x	2.91x	5.49x

Coalescing the output nearly doubles throughput on Pascal and is worth 4.6x on Ada. Coalescing the input alone is worth nothing at this block size on any card.

Combining the two is *worse* than output alone on Pascal — the input staging spends shared memory and a barrier, buys nothing, and pays in occupancy — but is the best variant on Ampere and Ada. The Pascal lesson did not transfer; it is recorded as architecture-specific.

At BLOCK=256 on Pascal the picture inverts: input staging is worth 4.4x, because the input no longer fits in L2. Same block size as the collapse in §5.2 — the second line of evidence for the L2 explanation.

With the optimised kernel the comparison was redone, not extrapolated. Best on the 1050 Ti: Huffman, BLOCK=64, 128 threads, output staged, **4.80 ms** — 2.02x faster than §5.2 — with the ladder at 5.07 ms (1.055x). The ladder now loses on both axes, in the regime where compute weighs more. One shared-memory lookup beats a dependent `clz/branch/shift/mask` chain.

5.4 The 8-bit block index

The observation. Block length has little spread. At BLOCK=64 on this data it runs from 130 to 273 bits: a range of 143, which fits in 8 bits.

The index. Instead of one absolute uint32 offset per block (4 B/block):

- one uint32 per *superblock* of 32 blocks — one warp — giving 0.125 B/block;
- one uint8 per block holding its length in bits, minus a global minimum — 1 B/block.

Total **1.125 B/block**. Each thread recovers its block’s offset as the superblock base plus an exclusive prefix-sum of the lengths of the preceding lanes: five `__shfl_up_sync` steps, no memory traffic. The builder raises an error rather than truncate if the range exceeds 255, which happens at BLOCK≥256 on this data.

The cost. BLOCK=64, 128 threads, output staged, Huffman:

	1050 Ti	A100	4090	compression (sample)
uint32 index	4.81 ms	0.47 ms	0.19 ms	30.73%
8-bit index + prefix-sum	4.82 ms	0.47 ms	0.19 ms	32.97%
time ratio	1.00x	1.006x	0.996x	+2.25 pts

Five shuffles are invisible next to a chain of 64 dependent loads. Index traffic falls from 4 MB to 1.1 MB.

What it changes. The reason to want large blocks was index overhead: at BLOCK=64 a uint32 index costs 0.5 bits per weight, at BLOCK=1024 0.03. Large blocks were 16x slower on Pascal

(§5.2) and, as §5.6 shows, 3–6x slower even on cards with no L2 cliff. The 8-bit index removes the dilemma. The whole-model rate at the fast configuration rises from 30.14% to 32.39%, within 0.68 points of the best any block size reaches, and there is no configuration on any of the three cards where a larger block is the right choice.

Against DFloat11’s index. DFloat11’s 5-bit gaps cost about 0.21 bits/weight on this data and require a counting pass before the decoding pass; the 8-bit scheme costs 0.14 (BLOCK=64) or 0.07 (BLOCK=128) and decodes once. Whether the single pass is faster than DFloat11’s two passes was not measured, since their kernel was not run here. Nothing about the scheme depends on the entropy code; it applies to any fixed-symbols-per-block stream whose block lengths have bounded spread.

Against the starting point on the same card (9.70 ms, 30.73%), the reference decoder is now 2.01x faster and 2.25 points smaller from two changes that touch neither the entropy code nor the decode loop.

5.5 The rate floor of the field split

Between neighbouring exponents there is nothing to exploit. Mutual information between adjacent exponents, in six 32M-symbol windows across the file: at most 0.010 bits, typically 0.0002. An order-1 context model gains 0.0000 bits/symbol. Huffman over pairs recovers 0.016 bits/symbol (0.1% of the file) for an 8 KiB table; the ladder over pairs is worse than the scalar ladder even with a 25x larger table.

Between the fields of one weight there is a little. Joint entropy is subadditive, so a coder over the whole 16-bit symbol — as in [2] — can never do worse than the field split, and does better by exactly the mutual information between fields. Measured exactly on all 596M unique weights (7,506 of the 65,536 possible values occur):

	bits/weight
$H(\text{sign}), H(\text{exp}), H(\text{mant})$	1.000, 2.645, 6.973
sum of marginals	10.618
$H(\text{sign}, \text{exp}, \text{mant})$ — true floor	10.578
$I(\text{exp}; \text{mant})$	0.040
$I(\text{sign}; \text{exp}), I(\text{sign}; \text{mant})$	0.0001, 0.0000

The dependence is concentrated. Conditional mantissa entropy is 7.00 bits for every exponent except the two largest binades, 122 and 123 (21% and 3.5% of the mass), where it falls to 6.84 and 6.26 bits: the Gaussian tail decays *within* the top binade, so small mantissas are more likely there.

What canonical Huffman achieves on each alphabet, no index:

	bits/weight
true floor $H(\text{sign}, \text{exp}, \text{mant})$	10.578
+ mutual information between fields	0.040
+ sign and mantissa stored raw, above their entropy	0.027
+ Huffman redundancy on the exponent alphabet	0.033
= field split, achieved	10.678
full 16-bit alphabet, achieved (floor + 0.024)	10.602

The field split gives away **0.076 bits/weight, 0.47 points**. Huffman on the exponent is 0.033 bits from its own floor; the design is 0.100 bits from the true floor of the symbol. That is the honest ceiling on any rate claim for exponent-only coding, and it bounds a full-alphabet coder’s rate advantage before any credit for a better entropy coder than Huffman.

5.6 The L2 prediction on two more cards

Written before renting the cards: *“On a card with a large L2 the BLOCK cliff should disappear; staging the input in shared should stop mattering at BLOCK=256; BLOCK=512–1024 should become viable. If the cliff is still there, the L2 explanation is false.”*

Base kernel, Huffman, 128 threads:

BLOCK	1050 Ti (85 B/thread)	A100 (190 B)	4090 (384 B)
64	9.70 ms	1.26 ms	1.24 ms
256	84.86	3.02	2.02
1024	157.16	2.46	2.13
1024 / 64	16.2x	1.96x	1.71x

The cliff prediction held. What remains is not L2: the floor kernel rises the same way, and at BLOCK=1024 there are only 62,500 threads for 221,184 (A100) or 196,608 (4090) resident slots.

The input-staging prediction failed, informatively. Its gain at BLOCK=256 is 4.4x, 2.0x and 1.2x on the three cards. It does not switch off when the data fits in L2; it decays with L2 per resident thread, because 32 scattered streams per warp are served more slowly from L2 than one contiguous load. L2 per thread predicts the *size* of the effect, not its presence.

“Large BLOCK becomes viable” is moot. With the optimised kernel, BLOCK=1024 is 3.5x (A100) and 6x (4090) slower than BLOCK=64: the serial chain is 16x longer and nothing hides it. And §5.4 removed the reason to want it.

Beyond the prediction. Best Huffman time for 64M symbols: 0.44 ms on the A100, 0.20 ms on the 4090. The 4090 is 2.2x faster with half the bandwidth, and its SM clock is 2.2x higher (2520 vs 1140 MHz). The base kernel shows no such effect (1.26 vs 1.24 ms: latency-bound on the scattered pattern). Achieved bandwidth at the best configuration is 16%, 10% and 45% of peak. Two cards do not prove it, but the reading most consistent with the data is that once the access pattern is fixed, the decoder is bound by the latency of its 64-step chain of dependent loads, which runs at SM clock.

The ladder-to-Huffman time ratio at the operating point is 1.05, ~1.0 and 1.3 on the three cards. On no card does the ladder win where it would be run.

6 Discussion

What was learned. The two structural changes were worth 2x in speed and 2.25 points. The entropy-code question the work was built on was worth 0.86 points, in the wrong direction. Entropy coding is finished *within the field split* — 0.033 bits from its floor, no neighbour correlation — and the split is now measured to cost 0.47 points, almost all in two binades. Everything else is structural: the access pattern, the index, the serial chain.

Relation to fused decompression [2]. The 11x gain over DFloat11 in [2] is attributed by its authors to fusion — no global-memory materialisation of the decompressed layer, decompression

overlapped with tensor-core work — not to rANS over Huffman. On this data the Shannon-gap argument for rANS is weak: Huffman is at 98.8% efficiency on the exponent alphabet.

The measurements here corroborate the structural reading from the other side. After every fix found here the standalone decoder is at 10–45% of peak bandwidth, its time follows SM clock, and its output — one byte per weight, then a second pass to merge sign and mantissa — is the majority of its traffic. Those are the costs fusion removes. The implication is to keep canonical Huffman and fuse. The interleaved-stream layout of [2], which obtains coalescing in the format rather than the kernel, maps onto §5.3. DFloat11’s kernel already stages and merges its output as §5.3 recommends; the corresponding step here is still to do.

Why lossy formats avoid entropy codes. Huffman coding of weights is old [3], but it targeted storage; GGUF, GPTQ, AWQ and NF4, which target inference, are fixed-width. The reason is not that Huffman is slow; it is that variable-length codes force a decompress-to-memory round trip instead of in-register dequantisation inside the GEMM. Losslessness closes the fixed-rate corner, so the index is not avoidable; making the decoder never touch DRAM is the reachable goal.

Threats to validity. One model, and a small one. One card per architecture, in rented containers whose clocks could not be locked (IQRs were zero). The GPU sample is the embedding matrix; both figures are given. No comparison against DFloat11’s shipped kernel — every “Huffman” number is this repository’s implementation of the same field split, and DFloat11’s two-pass, fixed-bytes-per-thread kernel is a different design whose speed relative to this one is unknown. No end-to-end tokens-per-second. No profiler data on any card. One unexplained anomaly on Pascal (Huffman 2–2.4x faster than the ladder at BLOCK=128 with input staged) is off the optimal path, reproduces, and is recorded rather than explained. The full list is in `METHODOLOGY.md`.

7 Conclusion

A table-free prefix code does not decode BF16 exponents faster than canonical Huffman on a GPU — on Pascal, Ampere or Ada — and costs 0.86 points of compression. The question that motivated the work has a clear negative answer.

The decoder’s time was never in the entropy code. It was in an access pattern that scattered each warp’s loads and stores; in an index four times larger than it needed to be; and, once those are fixed, in the serial chain of dependent loads inside each block, which runs at SM clock and which only fusion with the consumer of the weights can hide.

Of the three, the index is the result that travels. Eight-bit block lengths and a warp prefix-sum cost five shuffles per thread, save 2.9 bytes per block against a uint32 index and about a third against DFloat11’s gaps, and remove the reason to trade block size against compression. Any fixed-symbols-per-block stream can use it.

Canonical Huffman on the exponent is 0.033 bits from its floor, and the field split is 0.076 bits from the floor of the whole symbol. There is no rate left to find inside this design; the next step, if one is taken, is structural.

Reproducibility and authorship

The repository contains the code; setup scripts that download the model and refuse to proceed unless the kernels verify bit-exact; raw logs and environment records for all four machines under `results/`; and `METHODOLOGY.md`, covering data provenance, per-experiment samples, protocols

and threats to validity. Every table in this report is transcribed from a committed log. The two rented GPU runs cost about 1.7 USD in total and can be repeated by following `RENT_A_GPU.md`. Claude Code (Anthropic) was used extensively — to implement the codecs, kernels and harnesses, to run and interpret the CPU analysis, to review the literature, to drive the rented GPU runs, and to draft and revise the documentation and this report — under the author’s direction. It was also used to audit the repository against its own evidence before publication; that audit found and corrected a duplicated tensor in the whole-model figures and a literature claim that a paper summary asserted and the paper does not. Every commit carries a **Co-Authored-By** trailer and a session link. The hypothesis, the decisions about what to measure and what to conclude, and any remaining errors are the author’s.

References

- [1] T. Zhang, M. Hariri, S. Zhong, V. Chaudhary, Y. Sui, X. Hu, A. Shrivastava. 70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11). *NeurIPS 2025*. arXiv:2504.11651. <https://github.com/LeanModels/DFloat11>
- [2] H. Tan, Y. Chen, G. Alonso, W.-F. Wong, B. He. Approaching Shannon Bound with Lossless LLM Weight Compression. *ISCA 2026*. arXiv:2606.15789.
- [3] S. Han, H. Mao, W. J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *ICLR 2016*. arXiv:1510.00149.
- [4] D. A. Huffman. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE* 40(9), 1952.
- [5] E. S. Schwartz, B. Kallick. Generating a canonical prefix encoding. *Communications of the ACM* 7(3), 1964.
- [6] Qwen Team. Qwen3-0.6B. <https://huggingface.co/Qwen/Qwen3-0.6B>